



UPPSALA
UNIVERSITET

Bayesian Statistics and Data Analysis

Lecture 6

Måns Magnusson

Department of Statistics, Uppsala University
Thanks to Aki Vehtari, Aalto University

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Oral Exam 2

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Next Monday
- Block 3-5
- New room and longer (but still expect 20s+ depending on nervousness level)



UPPSALA
UNIVERSITET

Recap: Markov chain Monte Carlo

- The **transition distribution** T describes the next state given the current state.
- Construct T so that $p(\theta | y)$ is its **stationary distribution**.

- **MCMC recap**
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Recap: Markov chain Monte Carlo

- The **transition distribution** T describes the next state given the current state.
 - Construct T so that $p(\theta | y)$ is its **stationary distribution**.
 - Under suitable conditions, the chain approaches this distribution from its starting point.
- + A general approach when direct posterior sampling is difficult.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Recap: Markov chain Monte Carlo

- MCMC recap

- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- The **transition distribution** T describes the next state given the current state.
- Construct T so that $p(\theta | y)$ is its **stationary distribution**.
- Under suitable conditions, the chain approaches this distribution from its starting point.
- + A general approach when direct posterior sampling is difficult.
 - Draws are dependent: precision depends on the effective sample size (ESS), S_{eff} .
 - Assess convergence and the Monte Carlo standard error (MCSE) of each estimated summary.



- Gibbs sampling
 - Update one parameter (or a block) from its full conditional:

$$\theta_j \sim p(\theta_j | \theta_{-j}, y)$$

- + Often easy to construct when conditionals are available.
- Single-parameter updates can mix slowly under strong correlation.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Recap: Gibbs and Metropolis

- Gibbs sampling

- Update one parameter (or a block) from its full conditional:

$$\theta_j \sim p(\theta_j | \theta_{-j}, y)$$

- + Often easy to construct when conditionals are available.
- Single-parameter updates can mix slowly under strong correlation.

- Metropolis(-Hastings) sampling

- Propose a joint (or block) move from J .
- The proposal **plus acceptance or rejection** defines T ; on rejection, the next state equals the current state.

- **MCMC recap**

- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Recap: Gibbs and Metropolis

- MCMC recap

- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Gibbs sampling

- Update one parameter (or a block) from its full conditional:

$$\theta_j \sim p(\theta_j | \theta_{-j}, y)$$

- + Often easy to construct when conditionals are available.
- Single-parameter updates can mix slowly under strong correlation.

- Metropolis(-Hastings) sampling

- Propose a joint (or block) move from J .
- The proposal **plus acceptance or rejection** defines T ; on rejection, the next state equals the current state.
- + A well-chosen joint proposal can handle correlation efficiently.
- Efficiency depends on proposal shape and scale, especially in high dimensions.



UPPSALA
UNIVERSITET

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Section 2

Hamiltonian Monte Carlo



Why Hamiltonian Monte Carlo?

- MCMC recap
 - **Hamiltonian Monte Carlo**
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- Want to build an **efficient** Markov chain
 - We want to sample all components of θ jointly.
 - We know the **unnormalized** posterior $q(\theta|y) = Z \cdot p(\theta|y)$, where Z does not depend on θ .
 - Standard Hamiltonian Monte Carlo (HMC) needs continuous unknowns and a sufficiently smooth log density.
 - Can the **gradient** of $\log q(\theta|y)$ guide efficient proposals?



Why Hamiltonian Monte Carlo?

- MCMC recap
 - **Hamiltonian Monte Carlo**
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- Want to build an **efficient** Markov chain
 - We want to sample all components of θ jointly.
 - We know the **unnormalized** posterior $q(\theta|y) = Z \cdot p(\theta|y)$, where Z does not depend on θ .
 - Standard Hamiltonian Monte Carlo (HMC) needs continuous unknowns and a sufficiently smooth log density.
 - Can the **gradient** of $\log q(\theta|y)$ guide efficient proposals?
 - **Hamiltonian Monte Carlo!**



What is Hamiltonian Monte Carlo?

- Add momentum variables to our posterior (**canonical** distribution)

$$p(\theta, \psi|y) = p(\theta|y) \cdot p(\psi|\theta, y),$$

in practice we let $p(\psi|\theta, y) = p(\psi)$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



What is Hamiltonian Monte Carlo?

- Add momentum variables to our posterior (**canonical** distribution)

$$p(\theta, \psi|y) = p(\theta|y) \cdot p(\psi|\theta, y),$$

in practice we let $p(\psi|\theta, y) = p(\psi)$

- Idea from physics (mechanics):
 - θ : position
 - ψ : momentum

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

What is Hamiltonian Monte Carlo?

- Add momentum variables to our posterior (**canonical** distribution)

$$p(\theta, \psi|y) = p(\theta|y) \cdot p(\psi|\theta, y),$$

in practice we let $p(\psi|\theta, y) = p(\psi)$

- Idea from physics (mechanics):
 - θ : position
 - ψ : momentum
- Define the Hamiltonian as

$$H(\theta, \psi) = -\log(p(\psi)) - \log(p(\theta|y)) \quad (1)$$

$$= K(\psi) + V(\theta), \quad (2)$$

where $K(\psi)$ is the **kinetic** energy and $V(\theta)$ is the **potential** energy



Why can we add momentum?

- Choose any proper momentum density $p(\psi)$, independent of θ , and define the joint target

$$p(\theta, \psi \mid y) = p(\theta \mid y)p(\psi).$$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Why can we add momentum?

- Choose any proper momentum density $p(\psi)$, independent of θ , and define the joint target

$$p(\theta, \psi \mid y) = p(\theta \mid y)p(\psi).$$

- Its marginal distribution for θ is unchanged:

$$\begin{aligned}\int p(\theta, \psi \mid y) d\psi &= p(\theta \mid y) \int p(\psi) d\psi \\ &= p(\theta \mid y).\end{aligned}$$

- Hence we can sample the joint target and **keep only θ** .

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Why can we add momentum?

- Choose any proper momentum density $p(\psi)$, independent of θ , and define the joint target

$$p(\theta, \psi \mid y) = p(\theta \mid y)p(\psi).$$

- Its marginal distribution for θ is unchanged:

$$\begin{aligned}\int p(\theta, \psi \mid y) d\psi &= p(\theta \mid y) \int p(\psi) d\psi \\ &= p(\theta \mid y).\end{aligned}$$

- Hence we can sample the joint target and **keep only θ** .
- Refreshing $\psi \sim p(\psi)$ is a draw from its full conditional. It changes the trajectory without changing the target posterior.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



What is Hamiltonian Monte Carlo?

- Hamiltonian dynamics (**preserve energy**)

$$\frac{d\theta}{dt} = \frac{\partial H}{\partial \psi} = \frac{\partial K}{\partial \psi} \quad (3)$$

$$\frac{d\psi}{dt} = -\frac{\partial H}{\partial \theta} = -\frac{\partial V}{\partial \theta} \quad (4)$$

- For computing the dynamics, additive constants do not matter:

$$V(\theta) = -\log q(\theta | y) = -\log p(\theta) - \log p(y | \theta).$$

- Let $\psi \sim N(0, M)$, with positive-definite **mass matrix** M .

$$K(\psi) = -\log p(\psi) = \frac{1}{2}\psi^\top M^{-1}\psi + C,$$

where C does not depend on ψ .

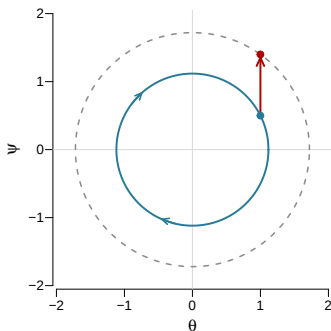
- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Normal example: constant energy

For $\theta \sim N(0, 1)$ and $\psi \sim N(0, 1)$ independently,

$$H(\theta, \psi) = \frac{\theta^2 + \psi^2}{2} \quad (\text{ignoring an additive constant}).$$



$$\frac{d\theta}{dt} = \psi, \quad \frac{d\psi}{dt} = -\theta.$$

- Exact dynamics follow a circle, clockwise: energy is constant.
- The red arrow refreshes momentum at fixed θ .
- The next trajectory can follow a different energy circle.



Choosing the mass matrix

- The mass matrix controls how momentum moves the parameters:

$$\frac{d\theta}{dt} = M^{-1}\psi.$$

- Write $\Sigma = \text{Cov}(\theta \mid y)$. For a roughly Gaussian posterior, a useful choice is

$$M^{-1} \approx \Sigma.$$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Choosing the mass matrix

- The mass matrix controls how momentum moves the parameters:

$$\frac{d\theta}{dt} = M^{-1}\psi.$$

- Write $\Sigma = \text{Cov}(\theta \mid y)$. For a roughly Gaussian posterior, a useful choice is

$$M^{-1} \approx \Sigma.$$

- During warmup, Stan estimates a regularized approximation:
 - diagonal: accounts for different parameter scales;
 - dense: also accounts for linear correlations.
- In Stan these calculations use **unconstrained coordinates**. One fixed matrix cannot remove all nonlinear geometry.



The leapfrog integrator

- A **discrete** approximation to Hamiltonian dynamics:

$$\frac{d\theta}{dt} = M^{-1}\psi, \quad \frac{d\psi}{dt} = \nabla_{\theta} \log q(\theta | y).$$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

The leapfrog integrator

- A **discrete** approximation to Hamiltonian dynamics:

$$\frac{d\psi}{dt} = M^{-1}\psi, \quad \frac{d\psi}{dt} = \nabla_{\theta} \log q(\theta | y).$$

- For $\ell = 0, \dots, L - 1$, repeat **all three updates**:

$$\psi_{\ell+1/2} = \psi_{\ell} + \frac{\epsilon}{2} \nabla_{\theta} \log q(\theta_{\ell} | y),$$

$$\theta_{\ell+1} = \theta_{\ell} + \epsilon M^{-1} \psi_{\ell+1/2},$$

$$\psi_{\ell+1} = \psi_{\ell+1/2} + \frac{\epsilon}{2} \nabla_{\theta} \log q(\theta_{\ell+1} | y).$$

- The final gradient uses the **updated position** $\theta_{\ell+1}$.



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

The leapfrog integrator

- A **discrete** approximation to Hamiltonian dynamics:

$$\frac{d\psi}{dt} = M^{-1}\psi, \quad \frac{d\psi}{dt} = \nabla_{\theta} \log q(\theta | y).$$

- For $\ell = 0, \dots, L - 1$, repeat **all three updates**:

$$\psi_{\ell+1/2} = \psi_{\ell} + \frac{\epsilon}{2} \nabla_{\theta} \log q(\theta_{\ell} | y),$$

$$\theta_{\ell+1} = \theta_{\ell} + \epsilon M^{-1} \psi_{\ell+1/2},$$

$$\psi_{\ell+1} = \psi_{\ell+1/2} + \frac{\epsilon}{2} \nabla_{\theta} \log q(\theta_{\ell+1} | y).$$

- The final gradient uses the **updated position** $\theta_{\ell+1}$.
- Step size ϵ controls numerical accuracy; $L\epsilon$ is the total integration time.
- Longer trajectories are not automatically harmless: errors can grow, especially when the step size is too large.



HMC: construct a proposal

At iteration t , start at θ_{t-1} with positive target density.

1. Refresh momentum:

$$\psi_0 \sim N(0, M).$$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



HMC: construct a proposal

At iteration t , start at θ_{t-1} with positive target density.

1. Refresh momentum:

$$\psi_0 \sim N(0, M).$$

2. Starting at (θ_{t-1}, ψ_0) , take L leapfrog steps of size ϵ to get $(\theta^*, \tilde{\psi})$.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



HMC: construct a proposal

At iteration t , start at θ_{t-1} with positive target density.

1. Refresh momentum:

$$\psi_0 \sim N(0, M).$$

2. Starting at (θ_{t-1}, ψ_0) , take L leapfrog steps of size ϵ to get $(\theta^*, \tilde{\psi})$.

3. Reverse the final momentum: $\psi^* = -\tilde{\psi}$. This makes the proposal reversible.

- For Gaussian momentum, $K(-\psi) = K(\psi)$, so the reversal does not change the energy or acceptance probability.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



HMC: accept or retain the old state

- Calculate the energy change:

$$\Delta H = H(\theta^*, \psi^*) - H(\theta_{t-1}, \psi_0).$$

- The Metropolis acceptance probability is

$$r = \min\left(1, \frac{q(\theta^* | y)p(\psi^*)}{q(\theta_{t-1} | y)p(\psi_0)}\right) = \min(1, e^{-\Delta H}).$$

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



HMC: accept or retain the old state

- Calculate the energy change:

$$\Delta H = H(\theta^*, \psi^*) - H(\theta_{t-1}, \psi_0).$$

- The Metropolis acceptance probability is

$$r = \min\left(1, \frac{q(\theta^* | y)p(\psi^*)}{q(\theta_{t-1} | y)p(\psi_0)}\right) = \min(1, e^{-\Delta H}).$$

- Draw $U \sim \text{Uniform}(0, 1)$ and compare on the log scale:

$$\theta_t = \begin{cases} \theta^*, & \log U < \min(0, -\Delta H), \\ \theta_{t-1}, & \text{otherwise.} \end{cases}$$

- As in L4, the log comparison avoids exponentiating extreme values. Discard the momentum; refresh it next iteration.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Numerical HMC: one leapfrog step

- Target $\theta \sim N(0, 1)$ and mass $M = 1$, so $\nabla_{\theta} \log q(\theta) = -\theta$.
- Suppose $\theta_{t-1} = 1$ and refreshed momentum $\psi_0 = 1$. Take one leapfrog step: $\epsilon = 1$, $L = 1$.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Numerical HMC: one leapfrog step

- Target $\theta \sim N(0, 1)$ and mass $M = 1$, so $\nabla_{\theta} \log q(\theta) = -\theta$.
- Suppose $\theta_{t-1} = 1$ and refreshed momentum $\psi_0 = 1$. Take one leapfrog step: $\epsilon = 1$, $L = 1$.

The momentum half-step, position full-step, and momentum half-step give

$$\psi_{1/2} = \psi_0 - \frac{\epsilon}{2} \theta_{t-1} = 1 - \frac{1}{2} \cdot 1 = 0.5,$$

$$\theta^* = \theta_{t-1} + \epsilon \psi_{1/2} = 1 + 1 \cdot 0.5 = 1.5,$$

$$\tilde{\psi} = \psi_{1/2} - \frac{\epsilon}{2} \theta^* = 0.5 - \frac{1}{2} \cdot 1.5 = -0.25.$$



Numerical HMC: one leapfrog step

- Target $\theta \sim N(0, 1)$ and mass $M = 1$, so $\nabla_{\theta} \log q(\theta) = -\theta$.
- Suppose $\theta_{t-1} = 1$ and refreshed momentum $\psi_0 = 1$. Take one leapfrog step: $\epsilon = 1$, $L = 1$.

The momentum half-step, position full-step, and momentum half-step give

$$\psi_{1/2} = \psi_0 - \frac{\epsilon}{2} \theta_{t-1} = 1 - \frac{1}{2} \cdot 1 = 0.5,$$

$$\theta^* = \theta_{t-1} + \epsilon \psi_{1/2} = 1 + 1 \cdot 0.5 = 1.5,$$

$$\tilde{\psi} = \psi_{1/2} - \frac{\epsilon}{2} \theta^* = 0.5 - \frac{1}{2} \cdot 1.5 = -0.25.$$

- Reverse momentum: $\psi^* = -\tilde{\psi} = 0.25$. The joint proposal is $(\theta^*, \psi^*) = (1.5, 0.25)$.
- Here ϵ and L are chosen for simple arithmetic.



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Numerical HMC: accept or reject

Ignoring an additive constant, $H(\theta, \psi) = (\theta^2 + \psi^2)/2$.

$$H(\theta_{t-1}, \psi_0) = \frac{1^2 + 1^2}{2} = 1,$$

$$H(\theta^*, \psi^*) = \frac{1.5^2 + 0.25^2}{2} = 1.15625.$$



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Numerical HMC: accept or reject

Ignoring an additive constant, $H(\theta, \psi) = (\theta^2 + \psi^2)/2$.

$$H(\theta_{t-1}, \psi_0) = \frac{1^2 + 1^2}{2} = 1,$$

$$H(\theta^*, \psi^*) = \frac{1.5^2 + 0.25^2}{2} = 1.15625.$$

- Leapfrog has increased the energy slightly:

$$\Delta H = 1.15625 - 1 = 0.15625.$$

- The acceptance probability is

$$r = \min(1, e^{-0.15625}) \approx 0.8553.$$



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Numerical HMC: accept or reject

Ignoring an additive constant, $H(\theta, \psi) = (\theta^2 + \psi^2)/2$.

$$H(\theta_{t-1}, \psi_0) = \frac{1^2 + 1^2}{2} = 1,$$

$$H(\theta^*, \psi^*) = \frac{1.5^2 + 0.25^2}{2} = 1.15625.$$

- Leapfrog has increased the energy slightly:

$$\Delta H = 1.15625 - 1 = 0.15625.$$

- The acceptance probability is

$$r = \min(1, e^{-0.15625}) \approx 0.8553.$$

- Draw $U \sim \text{Uniform}(0, 1)$. If $U = 0.80$, then $U < r$: **accept** and set $\theta_t = 1.5$.
- If instead $U = 0.90$, **reject** and retain $\theta_t = 1$. In either case, refresh momentum at the next iteration.



Pair discussion: step size and exploration

Discuss in pairs (1–2 minutes).

- Consider HMC with $\epsilon = 0.1$ and $L = 20$. We halve ϵ but keep L fixed.
- What happens to numerical accuracy and integration time? Does this automatically improve posterior exploration?
- How could we preserve the original integration time?

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Pair discussion: step size and exploration

Discuss in pairs (1–2 minutes).

- Consider HMC with $\epsilon = 0.1$ and $L = 20$. We halve ϵ but keep L fixed.
- What happens to numerical accuracy and integration time? Does this automatically improve posterior exploration?
- How could we preserve the original integration time?

After discussion:

- Smaller steps generally improve numerical accuracy, but $L\epsilon$ falls from 2 to 1.
- A shorter trajectory may limit movement, so better accuracy does not guarantee better exploration.
- Double L to 40 to keep $L\epsilon = 2$, at the cost of more gradient evaluations per proposal.



UPPSALA
UNIVERSITET

Why does the HMC update work?

- **Reversibility:** leapfrog followed by a momentum reversal can be undone by applying the same proposal again.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Why does the HMC update work?

- **Reversibility:** leapfrog followed by a momentum reversal can be undone by applying the same proposal again.
- **Volume preservation:** leapfrog does not expand or compress volumes in (θ, ψ) space. No proposal Jacobian is needed.

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Why does the HMC update work?

- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- **Reversibility**: leapfrog followed by a momentum reversal can be undone by applying the same proposal again.
- **Volume preservation**: leapfrog does not expand or compress volumes in (θ, ψ) space. No proposal Jacobian is needed.
- **Metropolis correction**: the acceptance step corrects the numerical energy error, preserving the joint target

$$p(\theta, \psi \mid y) \propto e^{-H(\theta, \psi)}.$$

- Marginalizing momentum then gives $p(\theta \mid y)$. Energy conservation alone is not the full justification.



UPPSALA
UNIVERSITET

Hamiltonian Monte Carlo

- Bivariate Normal HMC example

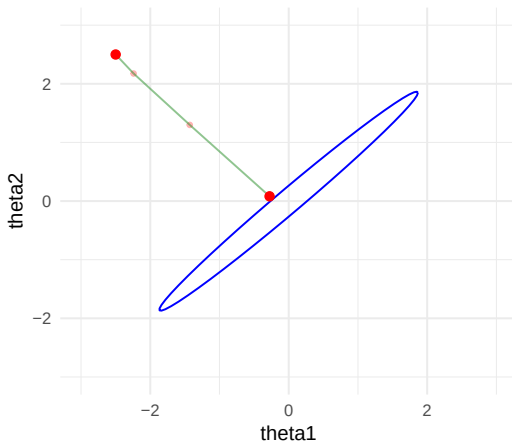
- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



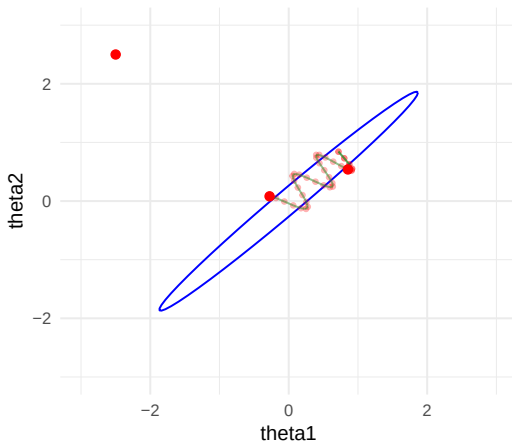
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



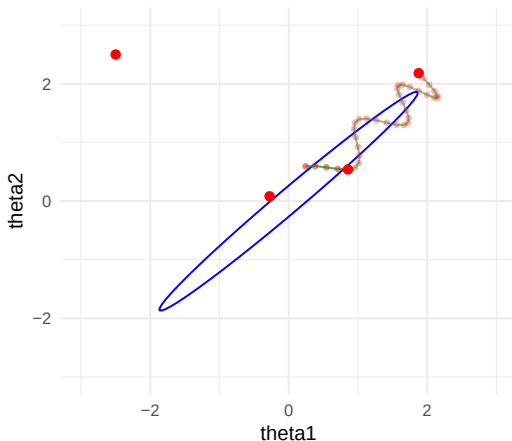
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



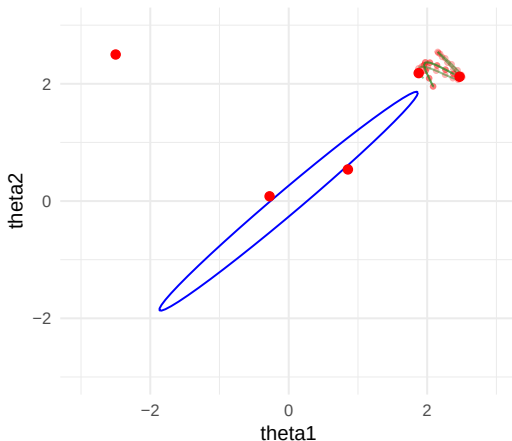
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



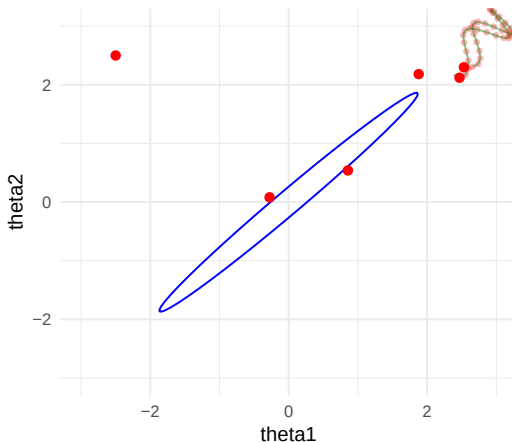
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



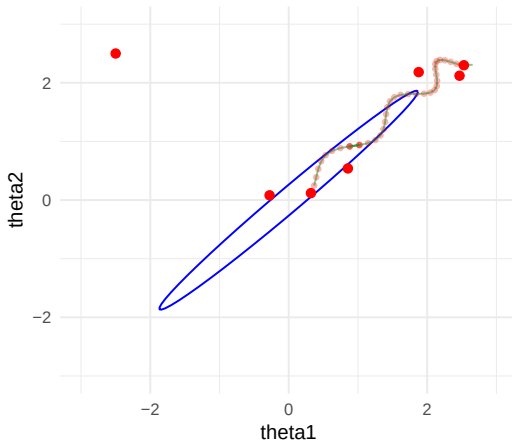
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



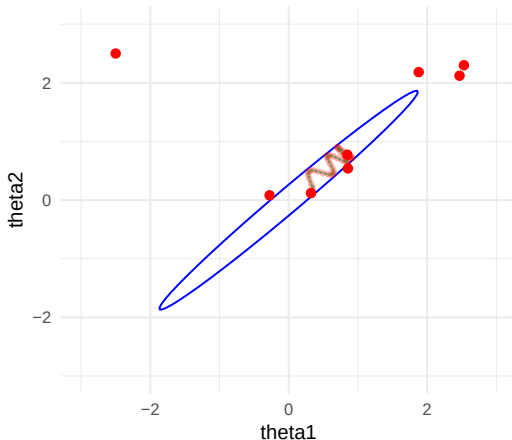
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



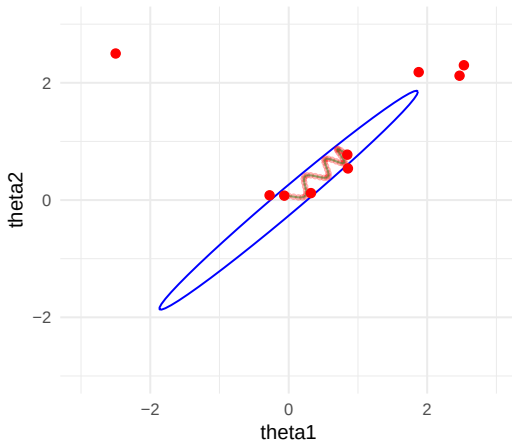
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example

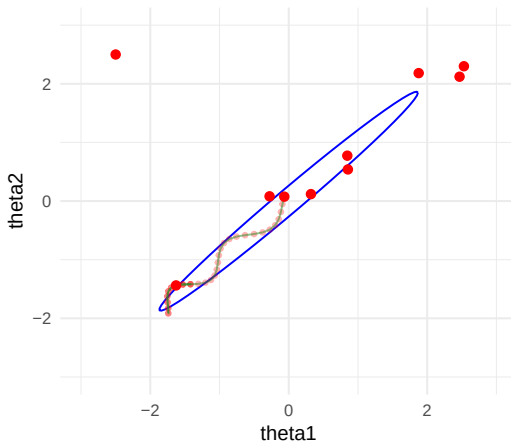


• Samples — Steps of the sampler — 90% HPD



Hamiltonian Monte Carlo

- Bivariate Normal HMC example



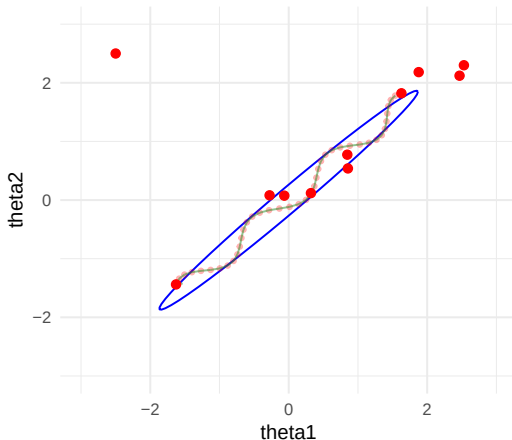
• Samples — Steps of the sampler — 90% HPD



- MCMC recap
- **Hamiltonian Monte Carlo**
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Hamiltonian Monte Carlo

- Bivariate Normal HMC example



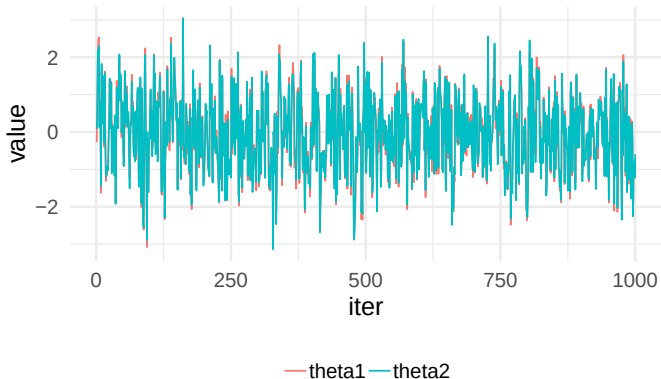
• Samples — Steps of the sampler — 90% HPD



Hamiltonian Monte Carlo

- Bivariate Normal HMC example

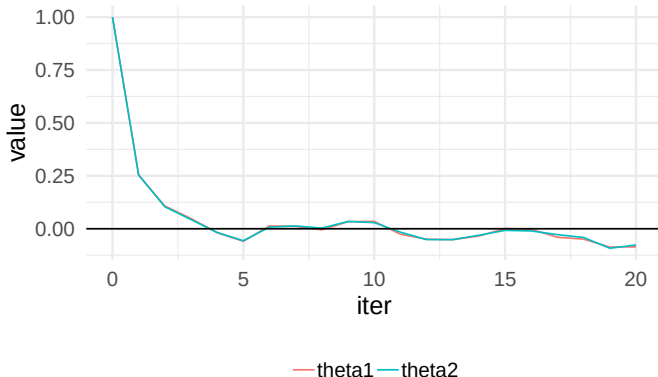
Trends





- Bivariate Normal HMC example

Autocorrelation function





UPPSALA
UNIVERSITET

Hamiltonian Monte Carlo summary

- Parameters:
 - step size

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Hamiltonian Monte Carlo summary

- Parameters:
 - ϵ step size
 - L leapfrog steps

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Hamiltonian Monte Carlo summary

- Parameters:
 - ϵ step size
 - L leapfrog steps
 - M mass matrix

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- Parameters:
 - ϵ step size
 - L leapfrog steps
 - M mass matrix
- + Can be very efficient (S_{eff})
- + Additional diagnostics

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Hamiltonian Monte Carlo summary

- Parameters:
 - ϵ step size
 - L leapfrog steps
 - M mass matrix
- + Can be very efficient (S_{eff})
- + Additional diagnostics
 - Integration time $L\epsilon$ needs tuning: too short mixes slowly; too long can retrace the trajectory.
- Requires continuous unknowns and a smooth log density.
- Stan transforms declared bounds to unconstrained coordinates.
- Discrete **observations** are fine; discrete **latent parameters** need another strategy, such as marginalization.
 - Each iteration requires many gradient evaluations.



Hamiltonian Monte Carlo summary

- Parameters:
 - ϵ step size
 - L leapfrog steps
 - M mass matrix
- + Can be very efficient (S_{eff})
- + Additional diagnostics
 - Integration time $L\epsilon$ needs tuning: too short mixes slowly; too long can retrace the trajectory.
- Requires continuous unknowns and a smooth log density.
- Stan transforms declared bounds to unconstrained coordinates.
- Discrete **observations** are fine; discrete **latent parameters** need another strategy, such as marginalization.
 - Each iteration requires many gradient evaluations.

demo



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Section 3

Dynamic HMC and NUTS



Dynamic HMC

- **Goal:** Simplify/adapt the tuning of HMC
- **Dynamic:** choose the number of leapfrog steps L within each iteration.
- The **No-U-Turn Sampler (NUTS)** grows a trajectory in randomly chosen forward or backward directions, doubling its length in stages.

- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Dynamic HMC

- **Goal:** Simplify/adapt the tuning of HMC
- **Dynamic:** choose the number of leapfrog steps L within each iteration.
- The **No-U-Turn Sampler (NUTS)** grows a trajectory in randomly chosen forward or backward directions, doubling its length in stages.
- **U-turn intuition**, with $M = I$ and one moving endpoint:

$$(\theta_L - \theta_{\text{start}})^\top \psi_L < 0.$$

The motion now points back towards the starting position.

- This is an illustration, **not the full stopping rule**: NUTS checks both ends and parts of the growing trajectory.



- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Dynamic HMC

- **Goal:** Simplify/adapt the tuning of HMC
- **Dynamic:** choose the number of leapfrog steps L within each iteration.
- The **No-U-Turn Sampler (NUTS)** grows a trajectory in randomly chosen forward or backward directions, doubling its length in stages.
- **U-turn intuition**, with $M = 1$ and one moving endpoint:

$$(\theta_L - \theta_{\text{start}})^\top \psi_L < 0.$$

The motion now points back towards the starting position.

- This is an illustration, **not the full stopping rule**: NUTS checks both ends and parts of the growing trajectory.
- Select the next state from eligible trajectory states using a rule designed to preserve the target distribution. These are not separately accepted MCMC draws.
- Step size still controls integration accuracy and computational cost.



UPPSALA
UNIVERSITET

Dynamic Hamiltonian Monte Carlo summary

- Parameters:

- step size

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Dynamic Hamiltonian Monte Carlo summary

- Parameters:

- ϵ step size

- M mass matrix

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Dynamic Hamiltonian Monte Carlo summary

- Parameters:

- ϵ step size

- M mass matrix

- + Can be very efficient (S_{eff})

- + Additional diagnostics

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Dynamic Hamiltonian Monte Carlo summary

- Parameters:

- ϵ step size

- M mass matrix

- + Can be very efficient (S_{eff})

- + Additional diagnostics

- L is chosen anew at each iteration, including after warmup.

- Stan transforms bounded parameters to unconstrained coordinates.

- Still requires continuous unknowns and a smooth log density. Discrete data are fine; discrete latent parameters need another strategy.

- Each iteration still requires many gradient evaluations.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Dynamic Hamiltonian Monte Carlo summary

- Parameters:

- ϵ step size

- M mass matrix

- + Can be very efficient (S_{eff})

- + Additional diagnostics

- L is chosen anew at each iteration, including after warmup.

- Stan transforms bounded parameters to unconstrained coordinates.

- Still requires continuous unknowns and a smooth log density. Discrete data are fine; discrete latent parameters need another strategy.

- Each iteration still requires many gradient evaluations.

[demo](#)



- **During warmup**, Stan adapts the mass matrix and step size.
 - The mass matrix accounts for parameter scales and, if dense, linear correlations.
 - `adapt_delta` is the target average acceptance statistic used to tune the step size.
 - A higher target usually gives smaller steps and more computation.

- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- **During warmup**, Stan adapts the mass matrix and step size.
 - The mass matrix accounts for parameter scales and, if dense, linear correlations.
 - `adapt_delta` is the target average acceptance statistic used to tune the step size.
 - A higher target usually gives smaller steps and more computation.
- `adapt_delta` is **not a guarantee** that this fraction of final NUTS draws will be accepted.



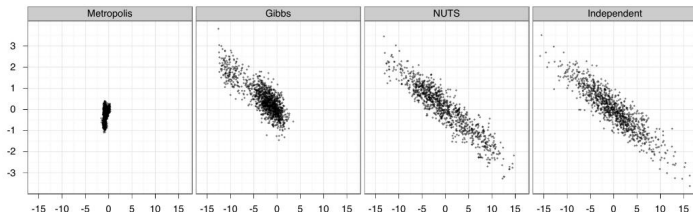
- MCMC recap
- Hamiltonian Monte Carlo
- **Dynamic HMC and NUTS**
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- **During warmup**, Stan adapts the mass matrix and step size.
 - The mass matrix accounts for parameter scales and, if dense, linear correlations.
 - `adapt_delta` is the target average acceptance statistic used to tune the step size.
 - A higher target usually gives smaller steps and more computation.
- `adapt_delta` is **not a guarantee** that this fraction of final NUTS draws will be accepted.
- **After warmup**, the adapted mass matrix and step size stop adapting.
 - NUTS still chooses a trajectory length L at **each iteration**.
 - `max_treedepth` limits the work per iteration.
- Use the post-warmup draws for inference.



Comparison of algorithms on **highly correlated** 250-dimensional Gaussian distribution

- Do **1,000,000** draws with both Random Walk Metropolis and Gibbs, thinning by 1000
- Do **1,000** draws using Stan's NUTS algorithm (no thinning)
- Do 1,000 independent draws (we can do this for multivariate normal)



Source: Jonah Gabry



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Section 4

HMC diagnostics



Maximum tree depth

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

- NUTS reached its computational limit before its usual stopping rule.
- Primarily an **efficiency warning**: repeated truncation can increase autocorrelation and reduce ESS.
- This alone does not demonstrate biased sampling or failure to explore the posterior.



Maximum tree depth

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

- NUTS reached its computational limit before its usual stopping rule.
- Primarily an **efficiency warning**: repeated truncation can increase autocorrelation and reduce ESS.
- This alone does not demonstrate biased sampling or failure to explore the posterior.
- Check model specification, scaling and parameterization; examine the other diagnostics too.
- Increasing `max_treedepth` permits more computation, but does not repair poor posterior geometry.



UPPSALA
UNIVERSITET

Divergent transitions

- HMC-specific diagnostic

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Divergent transitions

- HMC-specific diagnostic
- The Hamiltonian $H(\theta, \psi)$ should remain roughly constant throughout the trajectory:

$$H(\theta, \psi) \approx H(\theta_{\text{start}}, \psi_{\text{start}})$$

- Stan flags an excessively large positive increase at any point along the trajectory:

$$\Delta H = H(\theta, \psi) - H(\theta_{\text{start}}, \psi_{\text{start}}) \gg 0$$

- The numerical trajectory is unreliable, often because the posterior curvature changes rapidly.
- Divergences after warmup warn of possible biased estimates: some regions may be poorly explored.



- A step size that works in one region may be too large in another.
- In a highly curved region, small changes in position can produce large changes in the **gradient**.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Divergent transitions

- A step size that works in one region may be too large in another.
- In a highly curved region, small changes in position can produce large changes in the **gradient**.
- The gradient determines the **change in momentum**; it is not the momentum itself:

$$\frac{d\psi}{dt} = \nabla_{\theta} \log q(\theta | y).$$

- Narrow regions and changing scales, such as funnels in hierarchical models, can make leapfrog unstable.
- Strong linear correlation can often be handled by the mass matrix; changing local curvature is harder.



UPPSALA
UNIVERSITET

Divergent transitions

- When divergences remain after warmup:
 1. Check the model and code, parameter scales, and whether the priors provide sensible regularization.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



UPPSALA
UNIVERSITET

Divergent transitions

- When divergences remain after warmup:
 1. Check the model and code, parameter scales, and whether the priors provide sensible regularization.
 2. Reparameterize to make the geometry easier to traverse, for example with a non-centered hierarchical model.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



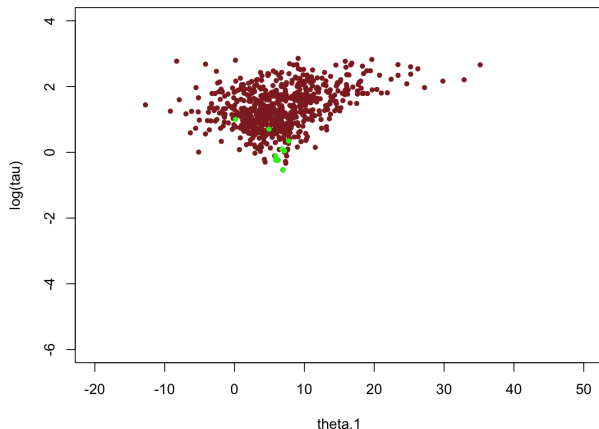
- When divergences remain after warmup:
 1. Check the model and code, parameter scales, and whether the priors provide sensible regularization.
 2. Reparameterize to make the geometry easier to traverse, for example with a non-centered hierarchical model.
 3. Consider a smaller step size. In Stan, a higher `adapt_delta` usually leads to smaller steps during warmup.
 4. Refit and check whether divergences disappear without making sampling impractically slow.
- More leapfrog steps **at the same step size** do not fix inaccurate integration. Longer chains do not remove this bias.



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Divergences (Betancourt, 2017)

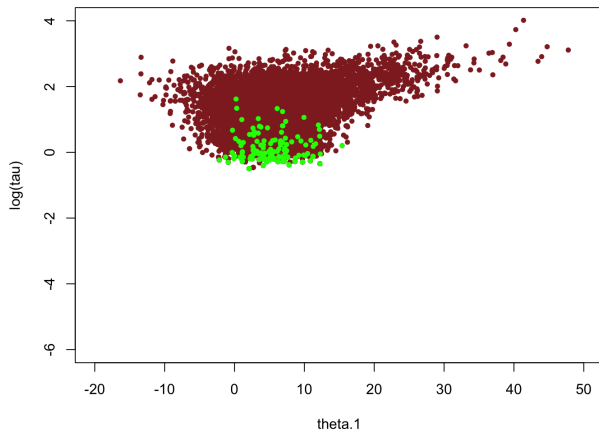




UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Divergences (Betancourt, 2017)

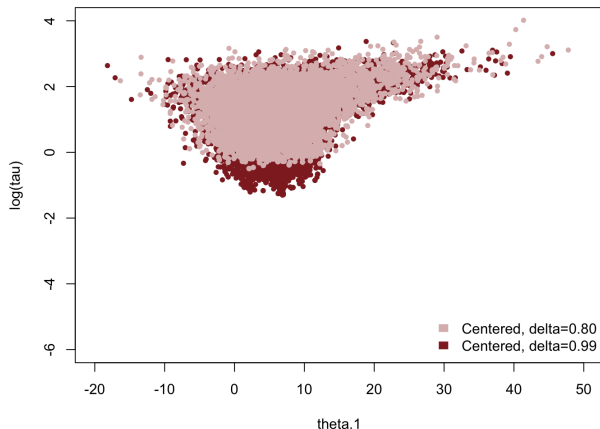




UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Divergences (Betancourt, 2017)

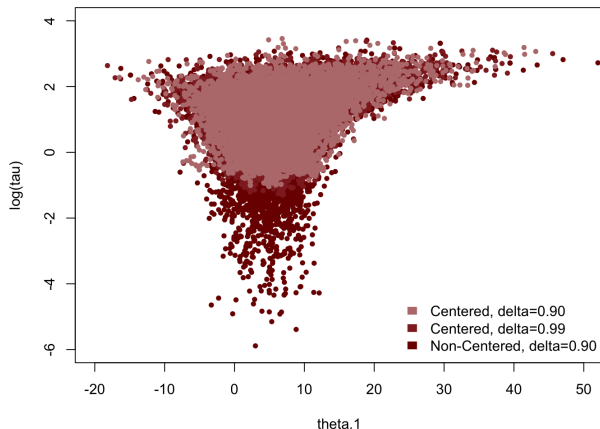




UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Divergences (Betancourt, 2017)





Difficult posterior geometry

- **Nonlinear dependence**: the direction of the narrow posterior region changes with position. A fixed mass matrix cannot straighten every part of it.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan



Difficult posterior geometry

- **Nonlinear dependence**: the direction of the narrow posterior region changes with position. A fixed mass matrix cannot straighten every part of it.
- **Funnels**: a step size suitable for the wide region may be too large in the narrow region.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Difficult posterior geometry

- **Nonlinear dependence**: the direction of the narrow posterior region changes with position. A fixed mass matrix cannot straighten every part of it.
- **Funnels**: a step size suitable for the wide region may be too large in the narrow region.
- **Separated modes**: HMC may also struggle to cross low-density regions between modes.

For comparison with L5, these demos use **random-walk Metropolis**:

Nonlinear dependence Funnel Multiple modes



- **Non-identifiability:** distinct parameter values imply the same data distribution; weak identification gives similar likelihoods.
 - With weak priors, this can produce posterior ridges or separated modes.
 - Priors can constrain these directions; a connected set of posterior modes is not guaranteed.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Identifiability and heavy tails

- **Non-identifiability**: distinct parameter values imply the same data distribution; weak identification gives similar likelihoods.
 - With weak priors, this can produce posterior ridges or separated modes.
 - Priors can constrain these directions; a connected set of posterior modes is not guaranteed.
- **Heavy tails**: the difficulty depends on the summary $f(\theta)$ being estimated.
 - If its posterior mean does not exist, there is no finite expectation to estimate.
 - Infinite variance rules out the usual finite-variance central limit theorem (CLT) and its MCSE formula. Other summaries may still be well behaved.

Ridge demo: random-walk Metropolis, as in L5



UPPSALA
UNIVERSITET

Diagnostic recap: can we trust the draws?

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Run multiple chains, preferably at least four, with dispersed initial values. Discard warmup.
- Inspect trace plots and require $\hat{R} < 1.01$ for the parameters and derived quantities of interest.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Diagnostic recap: can we trust the draws?

- Run multiple chains, preferably at least four, with dispersed initial values. Discard warmup.
- Inspect trace plots and require $\hat{R} < 1.01$ for the parameters and derived quantities of interest.
- Check ESS and MCSE **for each reported summary**. Using the classical ESS from L5, when the usual CLT applies,

$$\text{MCSE}(\hat{\mu}_f) \approx \frac{\text{SD}(f(\theta) \mid y)}{\sqrt{S_{\text{eff},f}}}.$$

- Investigate post-warmup divergences before trusting estimates. Tree-depth warnings primarily concern efficiency.



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Diagnostic recap: can we trust the draws?

- Run multiple chains, preferably at least four, with dispersed initial values. Discard warmup.
- Inspect trace plots and require $\hat{R} < 1.01$ for the parameters and derived quantities of interest.
- Check ESS and MCSE for each reported summary. Using the classical ESS from L5, when the usual CLT applies,

$$\text{MCSE}(\hat{\mu}_f) \approx \frac{\text{SD}(f(\theta) \mid y)}{\sqrt{S_{\text{eff},f}}}.$$

- Investigate post-warmup divergences before trusting estimates. Tree-depth warnings primarily concern efficiency.
- Good $\hat{R}$ and ESS do not cancel a divergence warning. No diagnostic proves that all important regions were explored.



Pair discussion: conflicting diagnostics

Discuss in pairs (1–2 minutes).

- Four chains give $\hat{R} = 1.00$ and $\text{ESS} = 2000$ for a parameter of interest, but report 12 post-warmup divergences.
- Can we trust its posterior summaries?
- Would simply running the chains longer solve the problem?

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- **HMC diagnostics**
- Probabilistic Programming
- Additional material
 - Stan

Pair discussion: conflicting diagnostics

Discuss in pairs (1–2 minutes).

- Four chains give $\hat{R} = 1.00$ and $\text{ESS} = 2000$ for a parameter of interest, but report 12 post-warmup divergences.
- Can we trust its posterior summaries?
- Would simply running the chains longer solve the problem?

After discussion:

- Good $\hat{R}$ and ESS do not rule out biased estimates when divergences remain.
- Longer chains do not repair inaccurate numerical integration or the underlying difficult geometry.
- Check the model, scales and priors. Consider reparameterization or smaller steps, then refit and check again.



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Section 5

Probabilistic Programming



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

The Box process: Probabilistic modeling

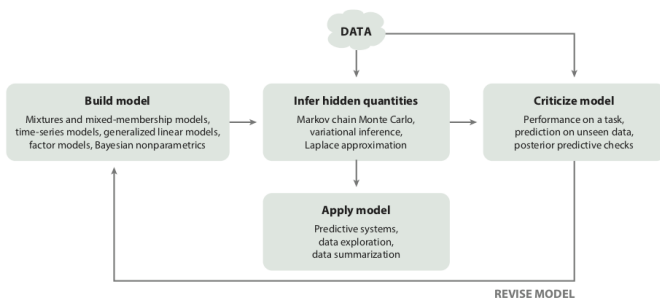


Figure: The Box approach (Box, 1976, Blei, 2014)



UPPSALA
UNIVERSITET

Probabilistic programming languages

- MCMC recap
 - Hamiltonian Monte Carlo
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- A probabilistic programming language (PPL) lets us specify a probability model in code and use inference algorithms to fit it.



- MCMC recap
 - Hamiltonian Monte Carlo
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- A probabilistic programming language (PPL) lets us specify a probability model in code and use inference algorithms to fit it.
 - To make probabilistic programming useful
 - easy workflow to build and revise models
 - inference has to be as automatic as possible
 - diagnostics for telling if the automatic inference doesn't work



UPPSALA
UNIVERSITET

Probabilistic programming

- MCMC recap
 - Hamiltonian Monte Carlo
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- Enables agile (incremental) workflow for developing probabilistic models
 - language
 - automated inference
 - diagnostics
 - Examples of frameworks: Stan, PyMC, Pyro, and NumPyro.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

From model to checked predictions

1. **Specify**: observation model, parameter constraints and scientifically meaningful priors.
2. **Fit**: run several chains with warmup; use a known-answer test when one is available.
3. **Check computation**: traces, $\hat{R}$, ESS, MCSE, divergences and tree depth. Resolve problems before interpreting results.
4. **Interpret and predict**: summarize parameters and generated quantities in meaningful units; distinguish the mean from a future outcome.
5. **Check the model**: compare predictive implications with observed data. Accurate computation does not guarantee a good model.

Assignment 6: apply this workflow to the bioassay model and the Gaussian trend/prediction exercise.



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- **Additional material**
 - Stan

Section 6

Additional material



UPPSALA
UNIVERSITET

Stan tutorial videos for this week

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Before Assignment 6, watch the short Stan tutorials:
 - [Getting started with Stan in R](#)
 - [Have I converged? Convergence checks in Stan](#)
- Python users may also watch [Getting started with Stan in Python](#)
- While watching, look for the roles of data, parameters, model, generated quantities, and the main diagnostics reported by Stan



UPPSALA
UNIVERSITET

Stan: probabilistic programming framework

- A modeling language with inference algorithms, diagnostics, documentation and worked examples.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



mc-stan.org



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Stan: probabilistic programming framework

- A modeling language with inference algorithms, diagnostics, documentation and worked examples.
- **Automatic differentiation** computes gradients of the log density for HMC.



mc-stan.org



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Stan: probabilistic programming framework

- A modeling language with inference algorithms, diagnostics, documentation and worked examples.
- **Automatic differentiation** computes gradients of the log density for HMC.
- Interfaces include RStan and CmdStanR for R, PyStan and CmdStanPy for Python, and the CmdStan command line.



mc-stan.org



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Stan: probabilistic programming framework

- A modeling language with inference algorithms, diagnostics, documentation and worked examples.
- **Automatic differentiation** computes gradients of the log density for HMC.
- Interfaces include RStan and CmdStanR for R, PyStan and CmdStanPy for Python, and the CmdStan command line.
- Packages such as `rstanarm` and `brms` provide formula-based model specification using Stan.



mc-stan.org



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Stanislaw Ulam (1909-1984)
 - Monte Carlo method
 - H-Bomb



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
data {  
  int<lower=0> N;      // number of experiments  
  int<lower=0,upper=N> y; // number of  
    successes  
}  
  
parameters {  
  // success probability  
  real<lower=0,upper=1> theta;  
}  
  
model {  
  theta ~ beta(1,1); //prior  
  y ~ binomial(N,theta); // observation model  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
data {  
  int<lower=0> N;      // number of experiments  
  int<lower=0,upper=N> y; // number of  
                        successes  
}  
  
parameters {  
  // success probability  
  real<lower=0,upper=1> theta;  
}  
  
model {  
  theta ~ beta(1,1); //prior  
  y ~ binomial(N,theta); // observation model  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
data {  
  int<lower=0> N;          // number of experiments  
  int<lower=0,upper=N> y;  // number of  
                           successes  
}  
  
parameters {  
  // success probability  
  real<lower=0,upper=1> theta;  
}  
  
model {  
  theta ~ beta(1,1);      //prior  
  y ~ binomial(N,theta);  // observation model  
}
```



```
data {  
  int<lower=0> N;           // trials  
  int<lower=0,upper=N> y;   // successes  
}
```

- Data type and size are declared
- Stan checks that supplied data match the declared types and constraints.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



```
data {  
  int<lower=0> N;           // trials  
  int<lower=0,upper=N> y;  // successes  
}
```

- Data type and size are declared
- Stan checks that supplied data match the declared types and constraints.
 - If you are not used to strong typing, this may feel annoying, but it will reduce the probability of coding errors, which will reduce probability of data analysis errors



```
parameters {  
  real<lower=0,upper=1> theta;  
}
```

- Stan samples an unconstrained variable z , then maps it back to the declared parameter θ .
- For the interiors of the declared bounds:

$$\text{<lower=a> : } z = \log(\theta - a),$$

$$\text{<lower=a,upper=b> : } z = \text{logit}\left(\frac{\theta - a}{b - a}\right).$$

- Here $a = 0$, $b = 1$, so $z = \text{logit}(\theta)$.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
parameters {  
  real<lower=0,upper=1> theta;  
}
```

- Stan samples an unconstrained variable z , then maps it back to the declared parameter θ .
- For the interiors of the declared bounds:

$$\text{<lower=a> : } z = \log(\theta - a),$$

$$\text{<lower=a,upper=b> : } z = \text{logit}\left(\frac{\theta - a}{b - a}\right).$$

- Here $a = 0$, $b = 1$, so $z = \text{logit}(\theta)$.
- Stan automatically adds the log-Jacobian adjustment for these **declared parameter constraints** (BDA3, p. 21).



```
model {  
  theta ~ beta(1,1);    // prior  
  y ~ binomial(N,theta); // likelihood  
}
```

- Each $\sim$ statement adds a log-density contribution, evaluated at the current parameter values.
- It **does not generate a random draw**: y remains the observed data; the sampler proposes values of θ .
- Together these statements specify $\log p(\theta) + \log p(y \mid \theta)$, up to constants.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
— Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
model {  
  theta ~ beta(1,1);    // prior  
  y ~ binomial(N,theta); // likelihood  
}
```

- Same target up to constants independent of the parameters:

```
model {  
  target += beta_lpdf(theta | 1, 1);  
  target += binomial_lpmf(y | N, theta);  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
model {  
  theta ~ beta(1,1);    // prior  
  y ~ binomial(N,theta); // likelihood  
}
```

- Same target up to constants independent of the parameters:

```
model {  
  target += beta_lpdf(theta | 1, 1);  
  target += binomial_lpmf(y | N, theta);  
}
```

- At each evaluation, target accumulates an **unnormalized log density**, not a normalized posterior.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
model {  
  theta ~ beta(1,1);    // prior  
  y ~ binomial(N,theta); // likelihood  
}
```

- Same target up to constants independent of the parameters:

```
model {  
  target += beta_lpdf(theta | 1, 1);  
  target += binomial_lpmf(y | N, theta);  
}
```

- At each evaluation, target accumulates an **unnormalized log density**, not a normalized posterior.
- For sampling on the unconstrained scale, it also includes the Jacobian adjustments from declared parameter constraints.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial model - Stan code

```
model {  
  theta ~ beta(1,1);    // prior  
  y ~ binomial(N,theta); // likelihood  
}
```

- Same target up to constants independent of the parameters:

```
model {  
  target += beta_lpdf(theta | 1, 1);  
  target += binomial_lpmf(y | N, theta);  
}
```

- At each evaluation, target accumulates an **unnormalized log density**, not a normalized posterior.
- For sampling on the unconstrained scale, it also includes the Jacobian adjustments from declared parameter constraints.
- `_lpdf`: continuous density; `_lpmf`: discrete probability mass. Prior and likelihood terms both contribute to the same target.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- We can run the same Stan code directly in the browser: stan-playground.flatironinstitute.org
- Useful for trying Stan syntax, changing priors, checking compiler messages and inspecting draws without installing Stan locally
- For the binomial example above, use this data in `data.json`:

```
{"N": 10, "y": 7}
```

- For assignments, still submit reproducible Stan and R/Python code as usual



Compiling a Stan model

- MCMC recap
 - Hamiltonian Monte Carlo
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- Stan translates the model to C++, which is then compiled for inference.
 - Compiled code evaluates the log density and its gradients.
 - The same Stan model can be used through different interfaces, including R and Python.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

RStan

```
library(rstan)
rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())

d_bin <- list(N = 10, y = 7)
fit_bin <- stan(
  file = 'binom.stan', data = d_bin,
  chains = 4, iter = 2000, warmup = 1000,
  seed = 123
)
```



RStan

```
library(rstan)
rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())

d_bin <- list(N = 10, y = 7)
fit_bin <- stan(
  file = 'binom.stan', data = d_bin,
  chains = 4, iter = 2000, warmup = 1000,
  seed = 123
)
```

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

PyStan 3

```
from pathlib import Path
import stan

code = Path('binom.stan').read_text()
data = dict(N=10, y=7)

posterior = stan.build(
    code, data=data, random_seed=123
)
fit = posterior.sample(
    num_chains=4, num_warmup=1000,
    num_samples=1000
)
```



UPPSALA
UNIVERSITET

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

PyStan 3

```
from pathlib import Path
import stan

code = Path('binom.stan').read_text()
data = dict(N=10, y=7)

posterior = stan.build(
    code, data=data, random_seed=123
)
fit = posterior.sample(
    num_chains=4, num_warmup=1000,
    num_samples=1000
)
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

1. Compile the model, unless a compiled version is available.
 2. Run warmup, including adaptation.
 3. Sample after warmup. For **each saved draw**:
 - evaluate generated quantities using that draw;
 - save the parameters and generated quantities together.
 4. Inspect diagnostics, including divergences, ESS and $\hat{R}$.
-
- Generated quantities are not evaluated just once at the end. They do not change the fitted posterior.



Binomial example: an exact benchmark

- In our example, $N = 10$, $y = 7$, and $\theta \sim \text{Beta}(1, 1)$.
- Bayes' rule gives

$$p(\theta \mid y) \propto \theta^7 (1 - \theta)^3 \cdot \theta^0 (1 - \theta)^0,$$
$$\theta \mid y \sim \text{Beta}(8, 4).$$

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Binomial example: an exact benchmark

- In our example, $N = 10$, $y = 7$, and $\theta \sim \text{Beta}(1, 1)$.
- Bayes' rule gives

$$p(\theta | y) \propto \theta^7 (1 - \theta)^3 \cdot \theta^0 (1 - \theta)^0,$$
$$\theta | y \sim \text{Beta}(8, 4).$$

- We know the posterior mean and standard deviation (SD) exactly:

$$E(\theta | y) = \frac{8}{12} = \frac{2}{3}, \quad \text{SD}(\theta | y) = \sqrt{\frac{8 \cdot 4}{12^2 \cdot 13}} \approx 0.1307.$$

- Stan's estimates should agree **within Monte Carlo error**, not match every digit.



Does Stan reproduce the known answer?

```
s_bin <- summary(fit_bin, pars = "theta",  
  probs = c(0.05, 0.95))$summary  
qbeta(c(0.05, 0.95), 8, 4)
```

	Exact	Stan
Posterior mean	0.6667	0.6648
Posterior SD	0.1307	0.1338
5% quantile	0.4356	0.4271
95% quantile	0.8649	0.8634

- RStan run above: 4000 post-warmup draws in total.
- MCSE of the mean is 0.0033; the mean differs by only 0.0019.
- $\hat{R} = 1.0009$; no divergences or maximum-tree-depth hits. Checking a known case helps validate code before harder models.



Difference between proportions

- MCMC recap
 - Hamiltonian Monte Carlo
 - Dynamic HMC and NUTS
 - HMC diagnostics
 - Probabilistic Programming
 - Additional material
 - Stan
- An experiment was performed to estimate the effect of beta-blockers on mortality of cardiac patients
 - A group of patients were randomly assigned to treatment and control groups:
 - out of 674 patients receiving the control, 39 died
 - out of 680 receiving the treatment, 22 died



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Difference between proportions

```
data {  
  int<lower=0> N1;  
  int<lower=0> y1;  
  int<lower=0> N2;  
  int<lower=0> y2;  
}  
parameters {  
  real<lower=0, upper=1> theta1;  
  real<lower=0, upper=1> theta2;  
}  
model {  
  theta1 ~ beta(1,1);  
  theta2 ~ beta(1,1);  
  y1 ~ binomial(N1, theta1);  
  y2 ~ binomial(N2, theta2);  
}  
  
generated quantities {  
  real oddsratio;  
  oddsratio = (theta2 / (1-theta2))  
              / (theta1 / (1-theta1));  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Difference between proportions

```
data {  
  int<lower=0> N1;  
  int<lower=0> y1;  
  int<lower=0> N2;  
  int<lower=0> y2;  
}  
parameters {  
  real<lower=0, upper=1> theta1;  
  real<lower=0, upper=1> theta2;  
}  
model {  
  theta1 ~ beta(1,1);  
  theta2 ~ beta(1,1);  
  y1 ~ binomial(N1, theta1);  
  y2 ~ binomial(N2, theta2);  
}  
  
generated quantities {  
  real oddsratio;  
  oddsratio = (theta2 / (1-theta2))  
              / (theta1 / (1-theta1));  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Difference between proportions

```
generated quantities {  
  real oddsratio;  
  oddsratio = (theta2 / (1-theta2))  
              / (theta1 / (1-theta1));  
}
```

- Evaluated **once for each saved draw** $(\theta_1^{(s)}, \theta_2^{(s)})$.
- The odds ratio is **deterministic given that draw**. Across draws it has a posterior distribution.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Difference between proportions

```
generated quantities {  
  real oddsratio;  
  oddsratio = (theta2 / (1-theta2))  
              / (theta1 / (1-theta1));  
}
```

- Evaluated **once for each saved draw** $(\theta_1^{(s)}, \theta_2^{(s)})$.
- The odds ratio is **deterministic given that draw**. Across draws it has a posterior distribution.
- An `_rng` function instead generates a new random outcome. For example, `binomial_rng(N2, theta2)` simulates a replicated death count for a group of size N_2 .
- Both use the fitted posterior; neither changes it.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Difference between proportions

```
d_bin2 <- list(  
  N1 = 674, y1 = 39, N2 = 680, y2 = 22  
)  
fit_bin2 <- stan(  
  file = 'binom2.stan', data = d_bin2,  
  chains = 4, iter = 2000, warmup = 1000,  
  seed = 123  
)
```

```
SAMPLING FOR MODEL 'binom2' (CHAIN 1).  
Iteration:    1 / 2000 [ 0%] (Warmup)  
...  
Iteration: 1000 / 2000 [ 50%] (Warmup)  
Iteration: 1001 / 2000 [ 50%] (Sampling)  
...  
Iteration: 2000 / 2000 [100%] (Sampling)
```

- Each chain retains 1000 draws after 1000 warmup iterations.
- Four chains give 4000 retained draws in total.



Difference between proportions

```
s <- summary(fit_bin2,  
             probs = c(0.1, 0.5, 0.9))$summary
```

	Mean	SD	10%	50%	90%
θ_1	0.0591	0.0091	0.0477	0.0587	0.0709
θ_2	0.0336	0.0070	0.0251	0.0330	0.0429
Odds ratio	0.5688	0.1573	0.3849	0.5504	0.7827

- Four chains, each with 1000 warmup iterations and 1000 retained draws.
- SD describes posterior uncertainty; the percentiles summarize the posterior distribution.



Monte Carlo precision of the summaries

```
s[, c("se_mean", "n_eff", "Rhat")]
```

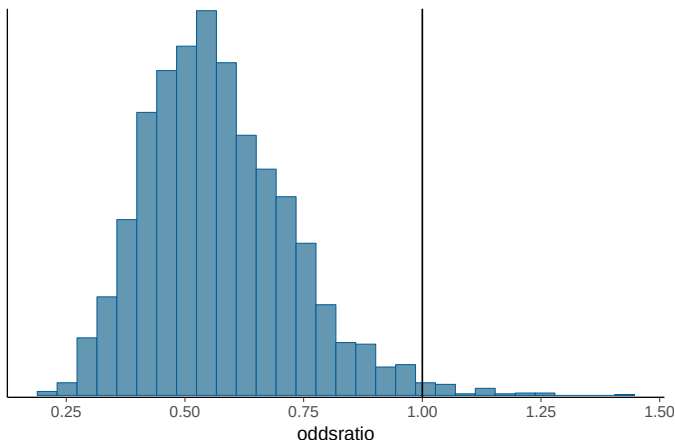
	MCSE of mean	ESS	$\hat{R}$
θ_1	0.00015	3856	0.9994
θ_2	0.00012	3604	0.9998
Odds ratio	0.00266	3505	0.9993
lp__	0.02423	1730	1.0005

- `se_mean` measures Monte Carlo error, not posterior uncertainty. Here we use classical ESS and split- $\hat{R}$.
- `monitor()` may report `warmup=0` because it receives draws with warmup already removed.
- `lp__` is the log density accumulated in target.



Difference between proportions

```
draws <- as.data.frame(fit_bin2)
breaks <- seq(0.25, 1.5, by = 0.25)
bayesplot::mcmc_hist(draws, pars = 'oddsratio') +
  ggplot2::geom_vline(xintercept = 1) +
  ggplot2::scale_x_continuous(breaks = breaks)
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

```
rstan :: check_hmc_diagnostics(fit_bin2)
```

- This run: 0 divergences and 0 maximum-tree-depth hits among 4000 retained draws.

```
n_leapfrog <-  
  rstan :: get_num_leapfrog_per_iteration(  
    fit_bin2  
  )
```



UPPSALA
UNIVERSITET

ShinyStan

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Graphical user interface for analysing MCMC results



Gaussian regression: the observation model

- At known predictor values x_i , assume conditional independence:

$$y_i \mid \alpha, \beta, \sigma, x_i \sim N(\alpha + \beta x_i, \sigma^2), \quad i = 1, \dots, N.$$

- α is the intercept, β the slope, and σ the residual **standard deviation**.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Gaussian regression: the observation model

- At known predictor values x_i , assume conditional independence:

$$y_i \mid \alpha, \beta, \sigma, x_i \sim N(\alpha + \beta x_i, \sigma^2), \quad i = 1, \dots, N.$$

- α is the intercept, β the slope, and σ the residual **standard deviation**.
- Set $\mu_i = \alpha + \beta x_i$. Stan's vectorized statement

`y ~ normal(mu, sigma);`

adds the sum of the individual log densities:

$$\log p(y \mid \alpha, \beta, \sigma, x) = \sum_{i=1}^N \log N(y_i \mid \mu_i, \sigma^2).$$

Stan takes σ , not σ^2 , as the second argument.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Gaussian linear model

```
data {  
  int<lower=0> N; // number of data points  
  vector[N] x; //  
  vector[N] y; //  
}  
parameters {  
  real alpha;  
  real beta;  
  real<lower=0> sigma;  
}  
transformed parameters {  
  vector[N] mu;  
  mu = alpha + beta*x;  
}  
model {  
  y ~ normal(mu, sigma);  
}
```

- This first version uses implicit flat priors. Proper priors are added on the following slides.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

```
data {  
    int <lower=0> N; // number of data points  
    vector [N] x; //  
    vector [N] y; //  
}
```

- difference between `vector[N]` `x` and `array[N]` `real x`



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Gaussian linear model

```
parameters {  
    real alpha;  
    real beta;  
    real<lower=0> sigma;  
}  
transformed parameters {  
    vector[N] mu;  
    mu = alpha + beta*x;  
}
```

- transformed parameters are deterministic transformations of parameters and data



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Priors for Gaussian linear model

```
data {  
  // N, x and y as before  
  real pmualpha;  
  real<lower=0> psalpha;  
  real pmubeta;  
  real<lower=0> psbeta;  
  real<lower=0> pssigma;  
}  
// parameters and mu as before  
model {  
  alpha ~ normal(pmualpha, psalpha);  
  beta ~ normal(pmubeta, psbeta);  
  sigma ~ normal(0, pssigma);  
  y ~ normal(mu, sigma);  
}
```

- Supply prior means and strictly positive scales in the units of the parameters.
- With $\sigma > 0$, its normal prior is a half-normal. The constraint alone does not specify this prior.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
— Stan

Gaussian regression: posterior prediction

Add `real xpred;` to the data block, then append:

```
generated quantities {  
  real mu_pred = alpha + beta*xpred;  
  real y_pred = normal_rng(mu_pred, sigma);  
}
```

- At each posterior draw, `mu_pred` is the regression mean at the fixed prediction location.
- `y_pred` adds a new residual to simulate a **future observation** at that location.
- Keep both: they answer different questions. These lines use the Gaussian observation model.



Mean uncertainty versus predictive uncertainty

At fixed x_{pred} , write $\mu_{\text{pred}} = \alpha + \beta x_{\text{pred}}$.

- The posterior of μ_{pred} describes uncertainty about the **regression mean**.
- A new observation also varies around that mean:

$$y_{\text{pred}} \mid \alpha, \beta, \sigma \sim N(\mu_{\text{pred}}, \sigma^2).$$

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan



Mean uncertainty versus predictive uncertainty

At fixed x_{pred} , write $\mu_{\text{pred}} = \alpha + \beta x_{\text{pred}}$.

- The posterior of μ_{pred} describes uncertainty about the **regression mean**.
- A new observation also varies around that mean:

$$y_{\text{pred}} \mid \alpha, \beta, \sigma \sim N(\mu_{\text{pred}}, \sigma^2).$$

- By the law of total variance, when these moments are finite,

$$\underbrace{\text{Var}(y_{\text{pred}} \mid y)}_{\text{predictive uncertainty}} = \underbrace{\text{Var}(\mu_{\text{pred}} \mid y)}_{\text{mean uncertainty}} + \underbrace{E(\sigma^2 \mid y)}_{\text{residual variation}}.$$

- A predictive interval for a new observation accounts for both sources, not just uncertainty in the fitted line.

Predictor values are held fixed throughout.

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
— Stan



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Student-t linear model

```
// Data and prior inputs as before
parameters {
  real alpha;
  real beta;
  real<lower=0> sigma;
  real<lower=1,upper=80> nu;
}
transformed parameters {
  vector[N] mu;
  mu = alpha + beta*x;
}
model {
  alpha ~ normal(pmualpha, psalpha);
  beta ~ normal(pmubeta, psbeta);
  sigma ~ normal(0, pssigma);
  nu ~ gamma(2, 0.1);
  y ~ student_t(nu, mu, sigma);
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- μ is the location and σ is a scale parameter, not generally the standard deviation.
- Smaller ν gives heavier tails; large ν approaches the Gaussian model.
- Here we restrict $1 < \nu < 80$:
 - the lower bound ensures a finite conditional mean;
 - the upper bound caps the range at nearly Gaussian errors.
- These bounds are modelling choices, not requirements of Stan.
- `gamma(2, 0.1)` uses shape 2 and rate 0.1. Together with the bounds, this defines a truncated gamma prior.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Standardized regression: data

```
data {  
  int<lower=2> N;  
  vector[N] x;  
  vector[N] y;  
  real xpred;  
}  
transformed data {  
  real mx = mean(x);  
  real my = mean(y);  
  real sx = sd(x);  
  real sy = sd(y);  
  vector[N] x_std;  
  vector[N] y_std;  
  if (sx <= 0 || sy <= 0)  
    reject("x and y must have positive SD");  
  x_std = (x - mx) / sx;  
  y_std = (y - my) / sy;  
}
```

- Center and scale once, using the training data. Model and prediction blocks follow on the next two slides.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Standardized regression: model

```
parameters {  
  real alpha_std;  
  real beta_std;  
  real<lower=0> sigma_std;  
}  
model {  
  alpha_std ~ normal(0, 1);  
  beta_std ~ normal(0, 1);  
  sigma_std ~ normal(0, 1);  
  y_std ~ normal(alpha_std + beta_std*x_std ,  
                  sigma_std);  
}
```

- β_{std} measures change in mean response SDs per predictor SD. σ_{std} is residual SD in response SDs.
- These are illustrative priors on the standardized scale. They need not match the earlier original-scale priors.



Standardized regression: original units

Expanding $y = m_y + s_y\{\alpha_{\text{std}} + \beta_{\text{std}}(x - m_x)/s_x + \text{error}\}$ gives

$$\beta = \frac{s_y}{s_x} \beta_{\text{std}}, \quad \alpha = m_y + s_y \alpha_{\text{std}} - \beta m_x, \quad \sigma = s_y \sigma_{\text{std}}.$$

```
generated quantities {  
  real beta = beta_std * sy / sx;  
  real alpha = my + sy*alpha_std - beta*mx;  
  real sigma = sy*sigma_std;  
  real mu_pred = alpha + beta*xpred;  
  real y_pred = normal_rng(mu_pred, sigma);  
}
```

- Transform **every posterior draw**, including the residual SD.
- xpred, mu_pred and y_pred are now in original units. Do not standardize the prediction point again.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- `rstanarm` uses R formulas and precompiled Stan models.
 - No separate model compilation for each fit.
 - Supports a defined collection of regression models.

Two-group binomial model:

```
d_bin2 <- data.frame(  
  N = c(674, 680), y = c(39, 22), grp2 = c(0, 1)  
)  
fit_bin2 <- stan_glm(  
  y/N ~ grp2, family = binomial(),  
  data = d_bin2, weights = N  
)
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- rstanarm uses R formulas and precompiled Stan models.
 - No separate model compilation for each fit.
 - Supports a defined collection of regression models.

Two-group binomial model:

```
d_bin2 <- data.frame(
  N = c(674, 680), y = c(39, 22), grp2 = c(0, 1)
)
fit_bin2 <- stan_glm(
  y/N ~ grp2, family = binomial(),
  data = d_bin2, weights = N
)
```

Gaussian linear model

```
fit_lin <- stan_glm(temp ~ year, data = d_lin)
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- **brms** uses R formulas to generate Stan code.
 - Supports many regression families and multilevel models.
 - Generated models need compilation before fitting.

```
fit_bin2 <- brm(  
  y | trials(N) ~ grp2,  
  family = binomial(), data = d_bin2  
)
```

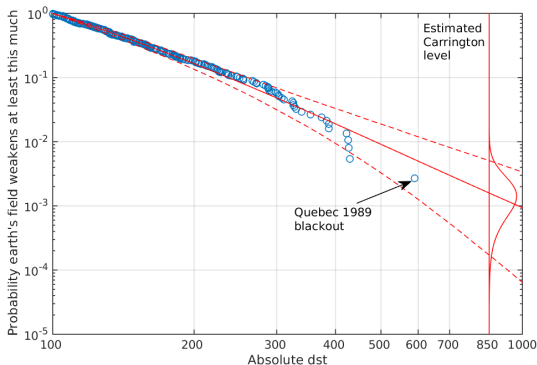
```
fit_lin_t <- brm(  
  temp ~ year, data = d_lin,  
  family = student()  
)
```



Geomagnetic storms

- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material

— Stan





- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Extreme value analysis

```
data {  
  int<lower=1> N;  
  vector<lower=0>[N] y;  
  int<lower=0> Nt;  
  vector<lower=0>[Nt] yt;  
}  
transformed data {  
  real ymax;  
  ymax = max(y);  
  if (ymax <= 0)  
    reject("At least one y must be positive");  
}  
parameters {  
  real<lower=0> sigma;  
  real<lower=-sigma/ymax> k;  
}  
model {  
  y ~ gpareto(k, sigma);  
}  
generated quantities {  
  vector[Nt] predccdf;  
  predccdf = gpareto_ccdf(yt, k, sigma);  
}
```



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
— Stan

Functions: generalized Pareto log density

```
functions {  
  real gpareto_lpdf(vector y,  
                    real k, real sigma) {  
    int N = num_elements(y);  
    if (sigma <= 0)  
      reject("sigma must be positive");  
    if (N == 0) return 0;  
    if (min(y) < 0 ||  
        (k < 0 && max(y) >= -sigma/k))  
      return negative_infinity();  
    if (k == 0)  
      return -sum(y)/sigma - N*log(sigma);  
    return -(1 + k)*sum(log1p(k*y/sigma))/k  
      - N*log(sigma);  
  }  
  // Add the survival function shown next here.  
}
```

- The density is zero outside the support, so its log is $-\infty$. For $k = 0$, the model is exponential.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Functions: generalized Pareto survival

```
// In the same functions block:
vector gpareto_ccdf(vector y,
                    real k, real sigma) {
  vector[num_elements(y)] s;
  if (sigma <= 0)
    reject("sigma must be positive");
  for (n in 1:num_elements(y)) {
    if (y[n] <= 0)
      s[n] = 1;
    else if (k < 0 && y[n] >= -sigma/k)
      s[n] = 0;
    else if (k == 0)
      s[n] = exp(-y[n]/sigma);
    else
      s[n] = exp(-log1p(k*y[n]/sigma)/k);
  }
  return s;
}
```

- Returns $P(Y > y_n \mid k, \sigma)$, not its logarithm.
- For $k < 0$, this probability is zero at and above the upper endpoint $-\sigma/k$.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- R
 - shinystan — interactive diagnostics
 - bayesplot — visualization and model checking (see model checking in Ch 6)
 - loo — cross-validation model assessment, comparison and averaging (see Ch 7)
 - projpred — projection predictive variable selection
- Python
 - ArviZ — visualization, and model checking and assessment (see Ch 6 and 7)



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- [RStan](#) and [PyStan](#) provide Stan interfaces for R and Python, respectively.
- [CmdStanR](#) and [CmdStanPy](#) run compiled models through CmdStan, Stan's command-line interface.
- The model is written in the same Stan language. Installation requirements and the commands for fitting and extracting draws differ.
- Related R packages: `posterior` for draws and diagnostics, and `tidybayes` and `ggdist` for summaries and visualization.



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

- Optional advanced talks after this lecture:
 - Bob Carpenter: Within-orbit adaptive leapfrog sampler (WALNUTS)
 - Adrian Seyboldt: From Mass Matrix Adaptation to Normalizing Flows
- You are not expected to understand every technical detail
- While watching, focus on:
 - What limitation of current HMC/NUTS is being discussed?
 - What new idea is proposed?
 - Why might this matter for difficult Bayesian models?



- MCMC recap
- Hamiltonian Monte Carlo
- Dynamic HMC and NUTS
- HMC diagnostics
- Probabilistic Programming
- Additional material
 - Stan

Stan references and tutorials

- Gelman, Lee and Guo (2015). [Stan: A probabilistic programming language for Bayesian inference and optimization](#). *Journal of Educational and Behavioral Statistics*, 40(5). [Author PDF](#).
- Carpenter et al. (2017). [Stan: A probabilistic programming language](#). *Journal of Statistical Software*, 76(1).
- Official documentation: [Stan User's Guide](#) (example models), [Stan Reference Manual](#), and [Stan Functions Reference](#).
- Gabry and Kennedy, StanCon 2019 Helsinki: Basics of Bayesian inference and Stan ([part 1](#), [part 2](#)).
- [Short practical Stan tutorials](#) (playlist).